RAPPORT FINAL DE PROJET

AuditHub

Plateforme de gestion documentaire des audits techniques

Nature :	Projet académique d'ingénierie logicielle
Technologies :	Django 5.2, React 19, SQLite, Playwright
Date du rapport :	15 octobre 2026
Version :	1.0
Classification :	CONFIDENTIEL — remise académique
Auteur / encadrant :	Informations non communiquées

Rapport fondé sur l'implémentation effectivement présente dans le dépôt.

Avertissement sur la nature du document

Le document de référence fourni adopte la structure d'un rapport d'audit de SMSI. Le présent livrable l'applique à l'évaluation technique du projet logiciel AuditHub. Il ne constitue ni un audit de certification ISO/IEC 27001, ni une recommandation de certification, ni une preuve de conformité d'une organisation. Les informations institutionnelles, identités, signatures et décisions qui ne figurent pas dans le dépôt sont explicitement signalées comme non vérifiables.

Table des matières

Avertissement sur la nature du document **i**

1 INFORMATIONS GÉNÉRALES **1**

Présentation du projet 1

1.3 Type d'audit 2

1.4 Dates et durée de l'audit 2

Problématique et besoins 2

2 OBJECTIFS DE L'AUDIT **3**

Objectifs fonctionnels 3

Objectifs de sécurité et de qualité 3

Critères de réussite retenus 3

3 PÉRIMÈTRE DE L'AUDIT **5**

3.2 Périmètre de l'audit (sites et processus audités) 5

Limites du périmètre 6

Interfaces et dépendances techniques 6

4 CRITÈRES D'AUDIT **7**

4.1 Norme de référence 7

4.2 Documents de référence complémentaires 7

4.3 Exigences internes du client 7

5 ÉQUIPE D'AUDIT **9**

Acteurs fonctionnels du système 9

6 MÉTHODOLOGIE D'AUDIT **11**

6.1 Approche générale 11

6.2 Techniques de collecte de preuves 11

Architecture technique 12
Architecture fonctionnelle et navigation 12
Modèle de données 12
Séquence d'une mutation authentifiée 13

7 DÉROULEMENT DE L'AUDIT 14
7.1 Chronologie 14
7.2 Réunions tenues 14
Implémentation backend 14
Implémentation frontend 15
Démonstration du flux d'audit 16
Sécurité implémentée 18

8 SYNTHÈSE DES CONSTATATIONS 20
8.1 Statistiques globales 20
8.2 Répartition par article / mesure 20
Points forts identifiés 21

9 NON-CONFORMITÉS DÉTAILLÉES 22
Position de la revue 22
ET-01 — persistance mono-instance 22
ET-02 — journal non inviolable 23
ET-03 — identité et récupération de compte 23
ET-04 — preuves uniquement textuelles 23
ET-05 — assurance de livraison partielle 24
ET-06 — exploitation à finaliser 24

10 OBSERVATIONS ET OPPORTUNITÉS D'AMÉLIORATION 25
O-01 — cohérence documentaire 25
O-02 — accessibilité et navigateurs 25
O-03 — score de risque 25
O-04 — intégration d'outils 25
BP-01 — défense en profondeur 26
BP-02 — preuves reproductibles 26

11 CONCLUSION ET RECOMMANDATION DE CERTIFICATION 27

11.1 Synthèse générale 27
Position relative à la certification 27
11.4 Prochaines ètapes 28

12 ANNEXES **29**
Annexe A : Liste des documents examinés 29
Annexe B : Inventaire du code examiné 29
Annexe C : Matrice des permissions 30
Annexe D : Endpoints principaux 30
Annexe E : Preuves documentaires collectées 31
Annexe F : Déclaration de non-responsabilité 31
Annexe G : Confidentialité 31
Signatures 31

Table des figures

5.1 Cas d'utilisation synthétiques déduits des permissions serveur 10

6.1 Architecture logique effectivement implémentée et cible de déploiement . 12
6.2 Flux principal d'une mission documentée 12
6.3 Relations principales du schéma de données 13
6.4 Séquence simplifiée d'une ècriture métier 13

7.1 Page de connexion réelle d'AuditHub 15
7.2 Tableau de bord alimenté par des données synthétiques 16
7.3 Création du périmètre et documentation d'un test 17
7.4 Détail d'une mission avec preuve textuelle et empreinte SHA-256 17
7.5 Historique daté d'un suivi de correction 18
7.6 Fonctions d'administration et de traçabilité 19

8.1 Matrice ISO affichée par l'application, explicitement présentée comme contribution 21

Liste des tableaux

1.1 Fiche d'identité technique d'AuditHub 1

2.1 Synthèse des exigences fonctionnelles vérifiables 4

3.1 Périmètre technique examiné 5

4.1 Sources de preuve et niveau de confiance 8

5.1 Rôles associés à la production et à la validation du rapport 9

6.1 Techniques appliquées au projet 11

7.1 Chronologie de la revue finale 14

8.1 Indicateurs objectifs de la version finale 20

8.2 Contribution technique aux mesures ciblées 21

9.1 Priorisation proposée des limites 24

11.1 Feuille de route recommandée 28

Chapitre 1

INFORMATIONS GÉNÉRALES

Présentation du projet

AuditHub est une application web monoproduit destinée à centraliser la documentation de missions de tests d'intrusion autorisés. Elle associe une interface React à une API JSON Django. Son rôle n'est pas d'exécuter Nmap, Metasploit, Burp Suite ou OWASP ZAP : les outils restent utilisés dans un laboratoire externe autorisé, puis leurs résultats sont consignés sous forme textuelle dans l'application.

Le besoin traité est la dispersion habituelle des informations d'un audit technique : autorisation, périmètre IP, intervenants, méthode, preuve, sévérité, responsable de correction, échéance et re-test. AuditHub rassemble ces objets dans une chaîne traçable et applique des contrôles d'accès côté serveur.

Table 1.1 – Fiche d'identité technique d'AuditHub

Rubrique	Information vérifiée
Backend	Django 5.2.17, vues fonctionnelles, formulaires et ORM
Frontend	React 19.3.0, bundle ES2020 produit par esbuild 0.28.2
Persistance	SQLite, migration applicative jusqu'à `0002_followup_constraints`
Authentification	Sessions Django, cookie HttpOnly, jeton CSRF, limitation des échecs
Rôles	Administrateur, Pentesteur, Auditeur
Tests	33 tests Django et 3 parcours Playwright validés le 15 octobre 2026
Déploiement documenté	Waitress derrière un reverse proxy TLS ; variables d'environnement obligatoires

1.3 Type d'audit

Le travail rapporté est une **revue technique et fonctionnelle du logiciel**, complétée par l'exécution de ses tests automatisés. Il ne s'agit pas d'un audit initial, de surveillance ou de renouvellement d'un SMSI au sens d'un organisme de certification. Cette qualification est essentielle : le dépôt ne contient ni mandat de certification, ni Déclaration d'Applicabilité organisationnelle, ni entretiens, ni preuves d'un environnement de production.

1.4 Dates et durée de l'audit

L'état final analysé et les validations présentées dans ce rapport datent du **15 octobre 2026**. La date de création du rapport de tests historique du projet est le 1er octobre 2026, puis il a èté actualisé. La durée humaine totale de conception et de développement n'est pas enregistrée dans le dépôt ; elle ne peut donc pas être chiffrée sans supposition.

Problématique et besoins

La problématique peut être formulée ainsi : *comment fournir une application simple, traçable et contrôlée pour documenter des tests de sécurité réalisés hors plateforme, sans transformer le serveur web en outil offensif ?* La réponse implémentée repose sur la séparation entre exécution et documentation, un périmètre explicite par mission, des rôles, la journalisation et un export JSON.

Chapitre 2

OBJECTIFS DE L'AUDIT

L'objectif de cette revue est d'évaluer si la version finale d'AuditHub correspond à son positionnement documenté et si ses principaux mécanismes sont cohérents avec le code, les migrations et les tests. Elle ne cherche pas à conclure sur l'efficacité globale d'un SMSI réel.

Objectifs fonctionnels

— vérifier la gestion de missions avec référence d'autorisation, dates, périmètre IP/CIDR et membres ;
— vérifier la saisie de constats issus de quatre familles d'outils, avec méthode, preuve, sévérité et correction ;
— examiner le suivi daté et l'historique des re-tests ;
— contrôler les exports de mission et du journal ;
— décrire les parcours React réellement disponibles.

Objectifs de sécurité et de qualité

La revue couvre l'authentification par session, les permissions fondées sur trois rôles, le cloisonnement par mission, CSRF, l'échappement des preuves affichées par React, les en-têtes HTTP, la limitation des tentatives de connexion, les contraintes de base et la gestion des secrets de configuration. Elle vérifie aussi la reproductibilité du build et l'existence d'une chaîne d'intégration continue.

Critères de réussite retenus

Une exigence est considérée comme mise en œuvre seulement lorsqu'elle est observable dans le code, la migration, l'interface ou un test. La simple présence d'un texte dans la documentation ne suffit pas. Inversement, les fonctions explicitement hors périmètre — exécution de scanners, import de fichiers, MFA, SSO ou certification — ne sont pas présentées comme livrées.

Table 2.1 – Synthèse des exigences fonctionnelles vérifiables

Exigence	Preuve d'implémentation	État
Authentification et session	Django Auth, endpoint `/api/session/`, cookie HttpOnly	Implémenté
RBAC et habilitations	décorateur `allowed`, groupes et filtre `visible_missions`	Implémenté
Mission autorisée	modèle `Mission`, formulaire et contrainte de dates	Implémenté
Constat technique	`TestRecord`, validation IP/date, SHA-256	Implémenté
Historique de suivi	`FollowupEvent`, API et export v2	Implémenté
Exécution des outils	aucune commande ni connecteur dans l'application	Hors périmètre
Import de rapports/fichiers	aucun téléversement ni parseur	Non implémenté

Chapitre 3

PÉRIMÈTRE DE L'AUDIT

3.2 Périmètre de l'audit (sites et processus audités)

Le terme « site » est interprété ici comme un composant du dépôt, aucun site physique n'ayant ètè visité. L'analyse couvre les sources et artefacts suivants.

Table 3.1 – Périmètre technique examiné

Zone	Emplacement	Contenu examiné
Configuration Django	`config/`	paramètres, middleware, URL, WSGI, sécurité de production
Métier backend	`portal/`	modèles, formulaires, vues, présentation JSON, permissions, commandes, migrations
Frontend	`frontend/src/`	navigation SPA, composants, formulaires dynamiques, client API, styles
Bundle servi	`static/react/`	JavaScript et CSS compilés, servis par Django
Tests	`portal/tests.py`, `frontend/tests/`	33 tests backend, 3 parcours de navigateur
Documentation	`README.md`, `docs/`	installation, architecture, guide auditeur, matrice ISO
Dépendance et CI	fichiers de verrouillage, `.github/`	versions, build et workflow automatisé

Les processus couverts sont l'ouverture de session, l'administration des comptes, la création d'une mission, l'affectation des membres, la saisie d'un test, la validation du périmètre, le suivi des corrections, la clôture/réouverture, la consultation du journal et l'export.

Limites du périmètre

La revue n'inclut pas de scan d'une cible, de test intrusif, d'inspection d'une infrastructure TLS réelle, d'analyse dynamique en production, ni d'évaluation humaine d'une organisation. SQLite est examiné comme choix du prototype ; aucune preuve de charge multi-utilisateur ou de haute disponibilité n'est disponible. Les captures utilisent exclusivement une base temporaire et des données synthétiques.

Interfaces et dépendances techniques

Le navigateur communique avec Django sur la même origine par des requêtes JSON sous `/api/`. La session reste dans un cookie HttpOnly ; le jeton CSRF fourni par `/api/session/` est envoyé dans l'en-tête `X-CSRFToken`. Le backend utilise l'ORM avec SQLite. En production documentée, Waitress reçoit les requêtes d'un reverse proxy TLS de confiance. Node.js n'est nécessaire qu'au build du frontend.

Chapitre 4

CRITÈRES D'AUDIT

Les critères retenus sont les exigences explicitement documentées par le projet, les comportements imposés par les formulaires et permissions, ainsi que les contrôles automatisés. Les normes ISO sont utilisées uniquement comme cadre de contribution, jamais comme base d'une déclaration de conformité.

4.1 Norme de référence

Le projet cible trois mesures mentionnées dans sa documentation : A.8.8 (gestion des vulnérabilités techniques), A.8.25 (cycle de développement sécurisé) et A.8.29 (tests de sécurité en développement et acceptation). Les pages publiques de l'ISO confirment le rôle général d'ISO/IEC 27001 et d'ISO/IEC 27002 [3, 4]. Les textes normatifs complets ne sont pas reproduits et n'ont pas ètè fournis dans le dépôt.

4.2 Documents de référence complémentaires

La revue s'appuie sur la documentation officielle Django relative à la sécurité, aux sessions, CSRF et au déploiement [2, 1], ainsi que sur la documentation React pour le rendu et l'échappement des valeurs [5]. Les fichiers propres au projet sont prioritaires pour décrire le comportement effectivement livré.

4.3 Exigences internes du client

Aucun client institutionnel ni corpus de politiques internes n'est identifiable. Les exigences internes utilisables sont donc celles du `README.md`, de `docs/ARCHITECTURE.md`, de `docs/GUIDE_AUDITEUR.md` et des tests. Cette limitation empêche toute conclusion sur des obligations contractuelles ou réglementaires propres à une organisation.

Table 4.1 – Sources de preuve et niveau de confiance

Source	Usage	Niveau
Code et migrations	comportement et structure de données	Èlevé
Tests exécutés	régressions couvertes et résultats datés	Èlevé dans le périmètre testé
Documentation du dépôt	intention, exploitation, limites déclarées	Moyen à èlevé
Captures automatisées	preuve visuelle des parcours synthétiques	Èlevé pour l'interface
Environnement de production	non fourni	Non vérifiable

Chapitre 5

ÉQUIPE D'AUDIT

L'identité de l'auteur, du professeur, d'un responsable d'audit ou d'un organisme de certification n'est pas présente dans le projet. Ces champs ne sont donc pas complétés artificiellement. Le tableau 5.1 distingue les responsabilités techniques observables des identités non communiquées.

Table 5.1 – Rôles associés à la production et à la validation du rapport

Rôle	Identité	Responsabilité
Auteur du projet	Non communiquée	conception, implémentation et remise académique
Professeur / èvaluateur	Non communiquée	èvaluation et validation académiques
Revue automatisée	Agent de documentation	lecture du dépôt, exécution des tests, captures et rédaction factuelle
Organisme de certification	Sans objet	aucune mission de certification n'est attestée

Acteurs fonctionnels du système

Les acteurs applicatifs ne doivent pas être confondus avec l'équipe de revue. Le code définit trois groupes :

Administrateur : accès à toutes les missions, création de missions et de comptes, gestion des rôles et de l'activation, clôture/réouverture, lecture et export du journal global ;

Pentesteur : consultation des missions affectées, saisie des résultats et suivi des corrections tant que la mission est active ;

Auditeur : lecture et export des missions affectées, accès à ses propres èvénements, sans modification métier.

Un superutilisateur Django est traité comme administrateur. Un compte authentifié sans groupe applicatif ne peut pas accéder aux fonctions métier.

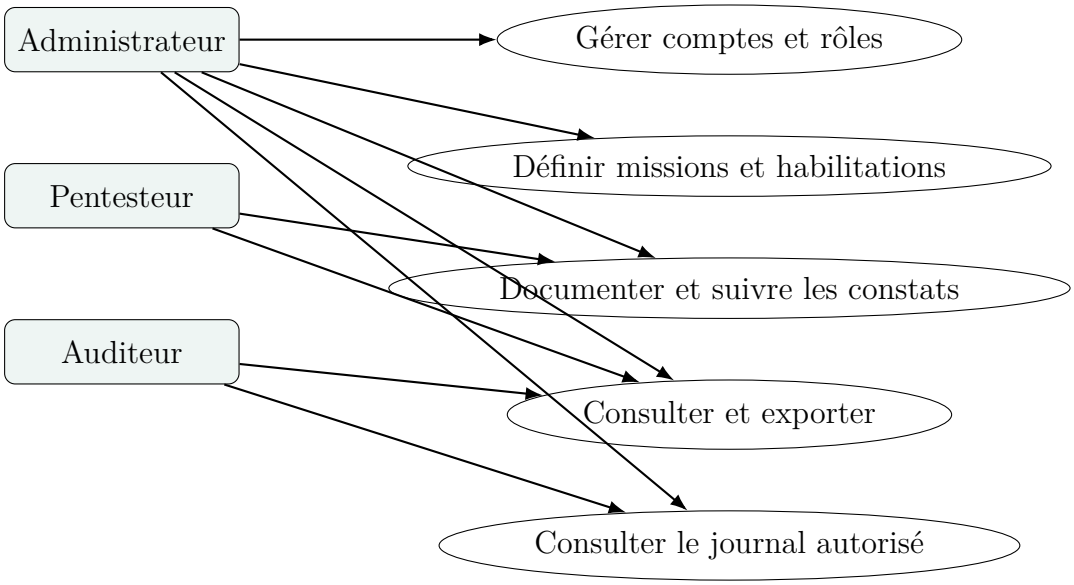


Figure 5.1 – Cas d'utilisation synthétiques déduits des permissions serveur

Chapitre 6

MÉTHODOLOGIE D'AUDIT

6.1 Approche générale

La méthode combine analyse statique, confrontation documentation–implémentation, exécution automatisée et inspection visuelle. Les affirmations sont classées en quatre catégories : implémenté, partiellement couvert, explicitement hors périmètre ou non vérifiable. Aucune propriété de sécurité n'est inférée d'une simple intention documentaire.

6.2 Techniques de collecte de preuves

Table 6.1 – Techniques appliquées au projet

Technique	Mise en œuvre	Preuves produites
Inventaire	parcours des fichiers hors dépendances générées	cartographie du dépôt
Revue du backend	lecture modèles, formulaires, vues, permissions, URL	exigences et flux métier
Revue du frontend	lecture client API, navigation, composants, CSS	pages et comportement SPA
Analyse des données	inspection des migrations et contraintes	modèle relationnel
Tests backend	`manage.pytest`	33 tests réussis
Tests navigateur	Playwright/Chrome headless, base temporaire	3 parcours réussis
Capture applicative	scénario Playwright dédié, données synthétiques	9 captures haute résolution
Contrôles de livraison	build React, checks Django, cohérence migrations	bundle et configuration valides

Architecture technique

L'architecture est volontairement monolithique et de même origine. Django sert la coque HTML, les fichiers statiques React et l'API. React pilote la navigation client et transforme les schémas de formulaire JSON en champs. Les règles métier demeurent côté serveur. L'ORM persiste dans SQLite. Cette architecture réduit les besoins CORS et évite le stockage d'un jeton d'accès dans le navigateur.

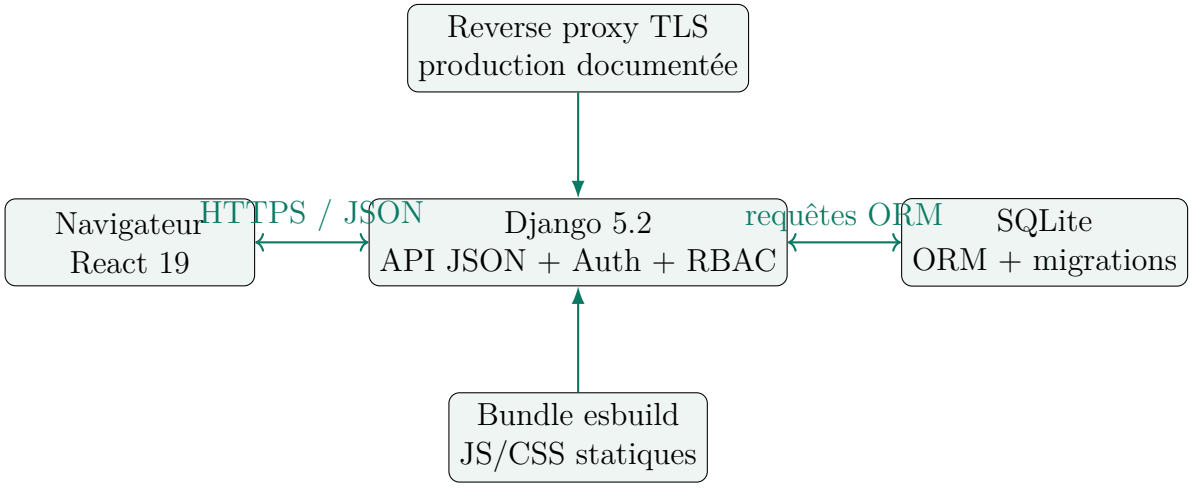


Figure 6.1 – Architecture logique effectivement implémentée et cible de déploiement

Architecture fonctionnelle et navigation

Le frontend ne recourt pas à un routeur externe. Il synchronise `window.history`, recharge la session et appelle le chemin API correspondant. Les pages disponibles sont la connexion, le tableau de bord, les missions, la bibliothèque d'outils, le journal, la matrice ISO et, pour l'administrateur, les utilisateurs. Les mutations utilisent `FormData` ; les erreurs de validation reviennent sous forme d'un schéma JSON et sont rendues au niveau des champs.

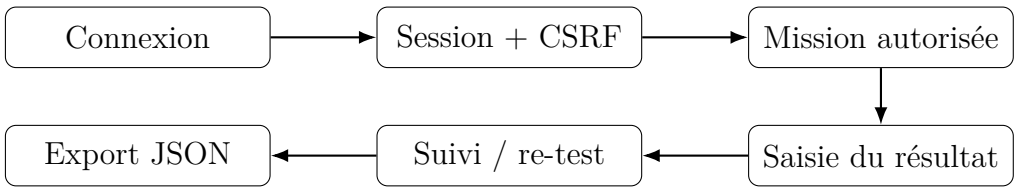


Figure 6.2 – Flux principal d'une mission documentée

Modèle de données

Le modèle s'appuie sur l'utilisateur et les groupes Django. Une mission est créée par un utilisateur et référence plusieurs membres. Chaque résultat appartient à une mission et conserve son auteur. Chaque mise à jour de suivi crée un `FollowupEvent` protégé contre la suppression en cascade. `AuditEvent` est un journal applicatif autonome ; `LoginAttempt` porte les fenêtres de limitation.

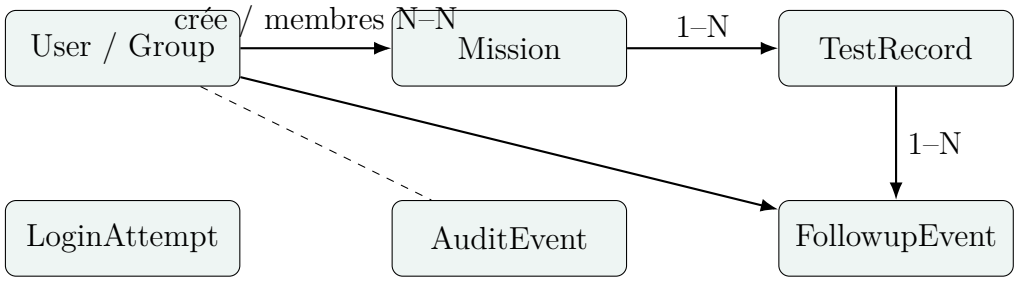


Figure 6.3 – Relations principales du schéma de données

Séquence d'une mutation authentifiée

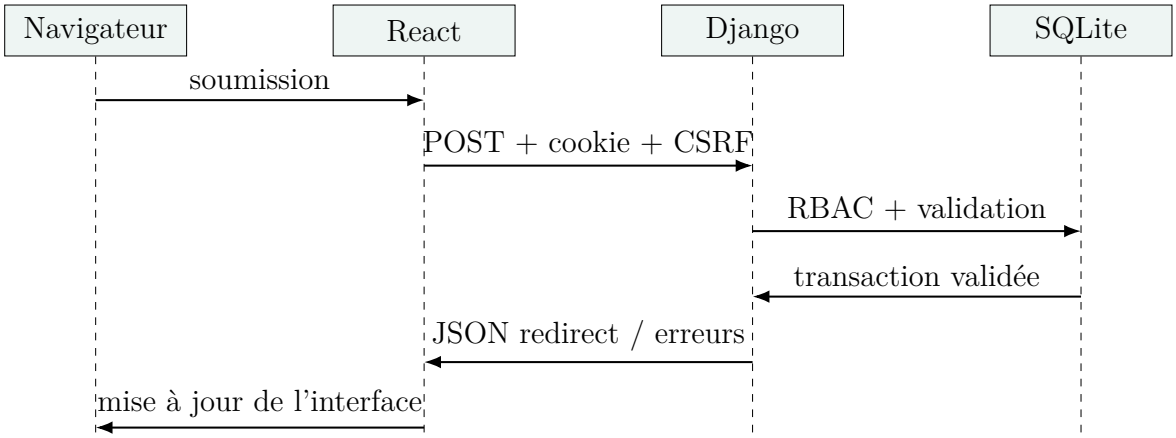


Figure 6.4 – Séquence simplifiée d'une ècriture métier

Chapitre 7

DÉROULEMENT DE L'AUDIT

7.1 Chronologie

Table 7.1 – Chronologie de la revue finale

Date	Activité	Résultat
1 oct. 2026	validation historique documentée	26 tests backend et 3 parcours navigateur dans la version d'alors
15 oct. 2026	revue finale du backend et correctifs préexistants au rapport	migration 0002, historique de suivi, proxy, RBAC administrateur
15 oct. 2026	validation automatisée	33 tests backend, 3 parcours navigateur, build React
15 oct. 2026	campagne de captures isolée	9 captures réelles avec données synthétiques
15 oct. 2026	production documentaire	sources LaTeX, diagrammes, bibliographie et checklist

7.2 Réunions tenues

Aucune réunion d'ouverture, intermédiaire ou de clôture n'est attestée par le dépôt. La validation s'est faite par analyse des artefacts et exécution automatisée. Les participants et comptes rendus ne doivent donc pas être inventés.

Implémentation backend

Les vues Django exposent les mêmes parcours sous le préfixe `/api/`. Le module `portal.api` sérialise utilisateurs, missions, constats, journaux, formulaires et pages paginées. Les erreurs non JSON sont normalisées par middleware pour l'API. Les écritures de mission, constat, suivi, changement d'état et administration des accès sont transactionnelles.

Les formulaires assurent la normalisation des réseaux avec `ipaddress`, limitent le périmètre à 100 entrées, refusent une cible hors réseau autorisé, contrôlent la date du test et imposent un commentaire de re-test pour l'état résolu. Des contraintes SQL empêchent une mission dont la fin précède le début et un constat résolu sans re-test.

Implémentation frontend

Le frontend est réparti entre `main.jsx` (état global, navigation et pages), `components.jsx` (composants réutilisables), `api.js` (requêtes et gestion des erreurs) et `style.css`. Le composant de formulaire dynamique interprète les champs fournis par Django, conserve les valeurs lors d'une erreur 422 et expose les messages via des attributs ARIA.

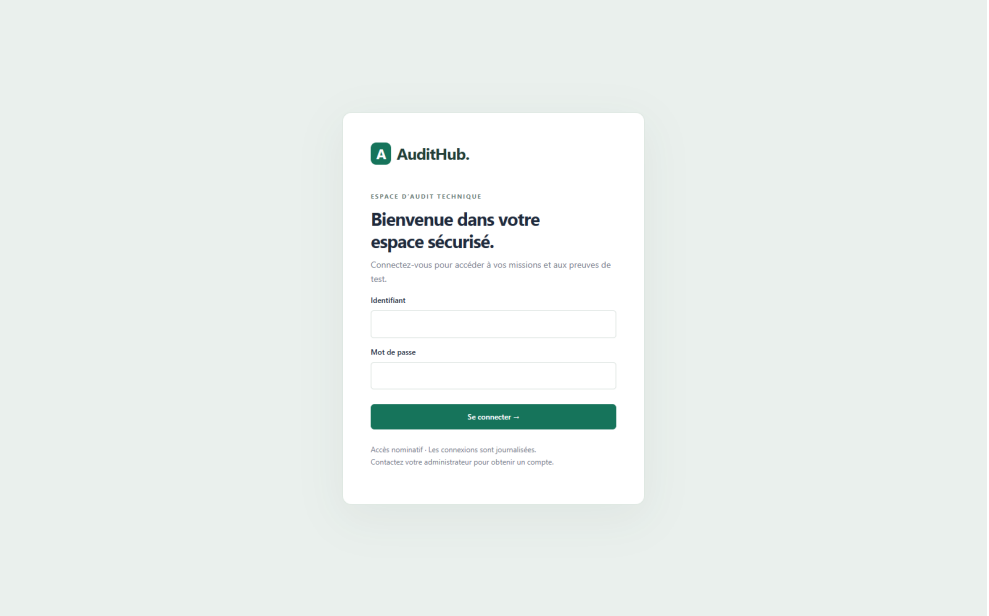


Figure 7.1 – Page de connexion réelle d'AuditHub

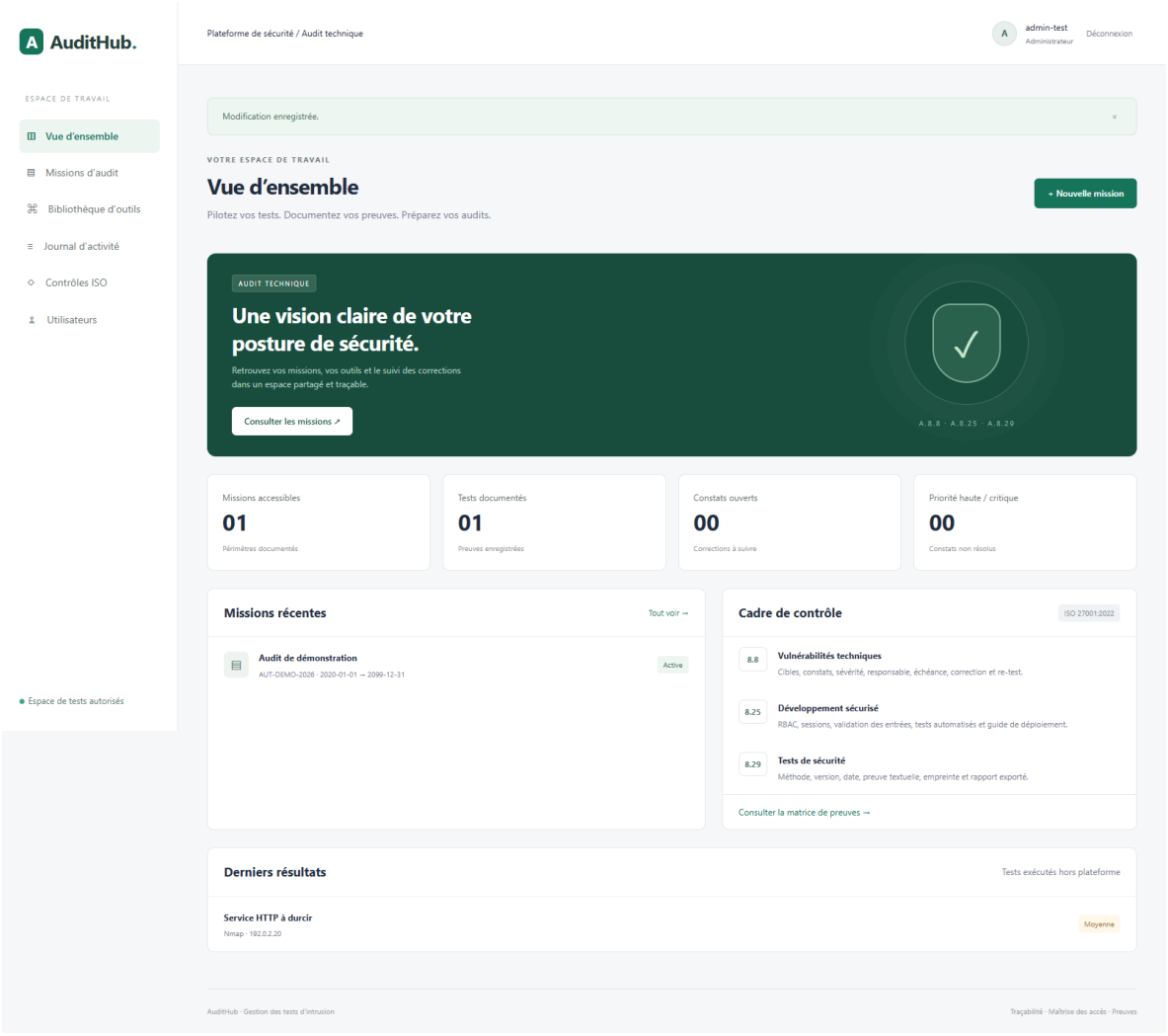
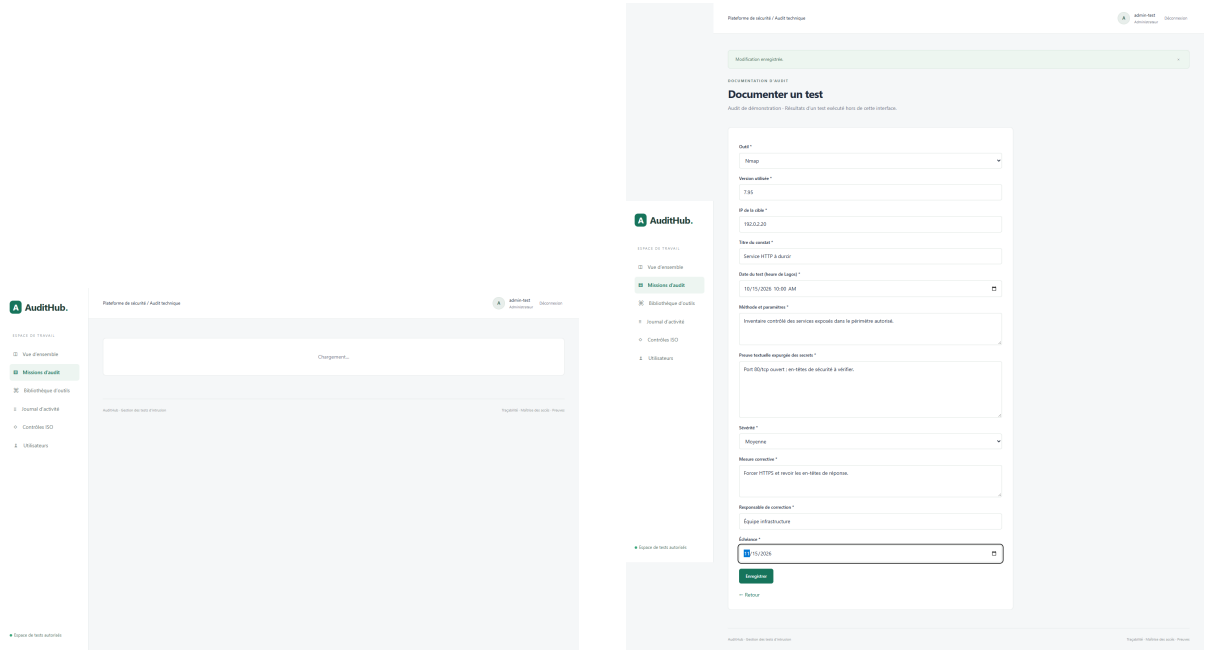


Figure 7.2 – Tableau de bord alimenté par des données synthétiques

Démonstration du flux d'audit

La campagne de capture crée une mission « Audit de démonstration » dans une base temporaire, lui associe un périmètre IPv4/IPv6, consigne un constat Nmap, puis enregistre un re-test. Les informations sont synthétiques et n'exposent aucune donnée de la base de travail.



(a) Formulaire de mission **(b)** Saisie d'un résultat

Figure 7.3 – Création du périmètre et documentation d'un test

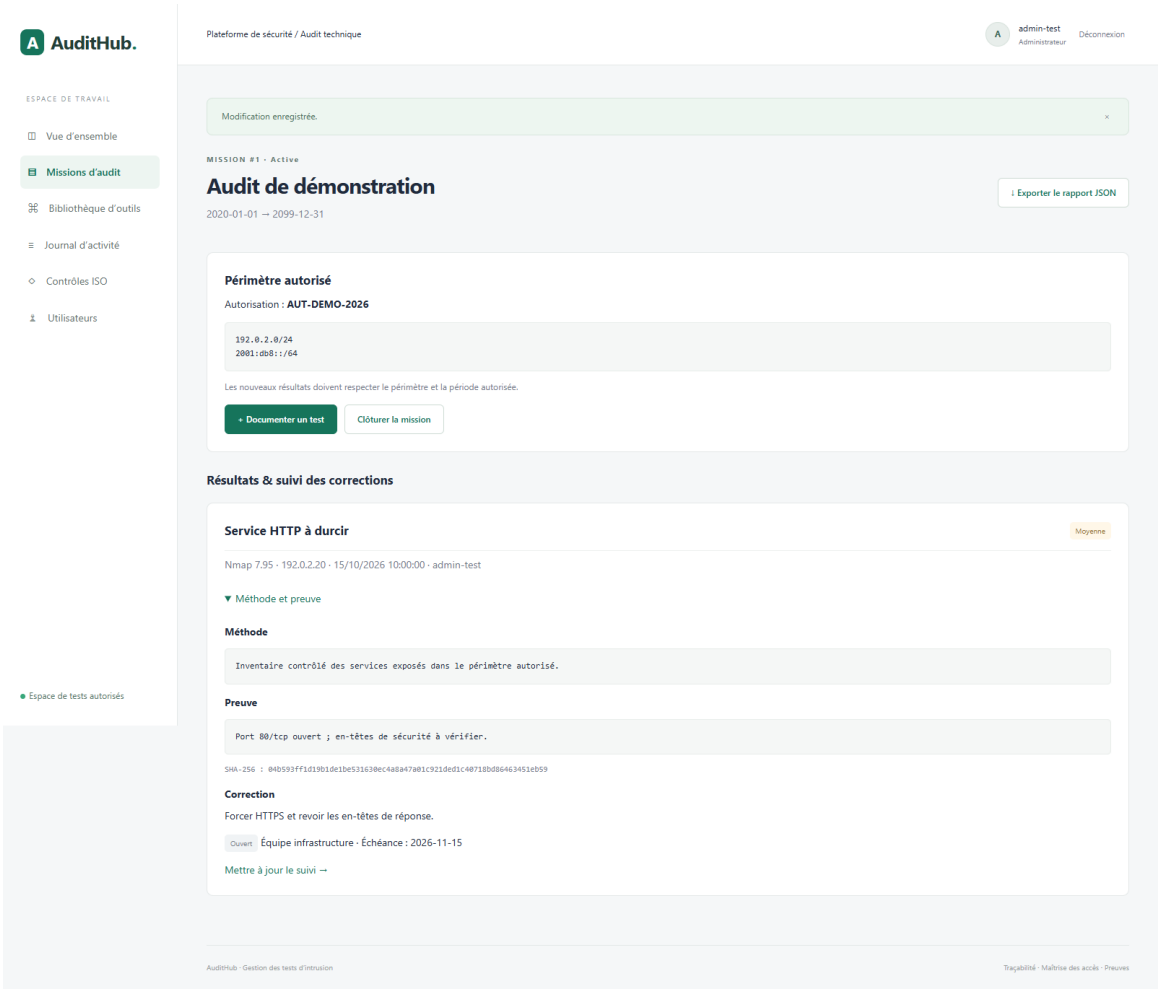


Figure 7.4 – Détail d'une mission avec preuve textuelle et empreinte SHA-256

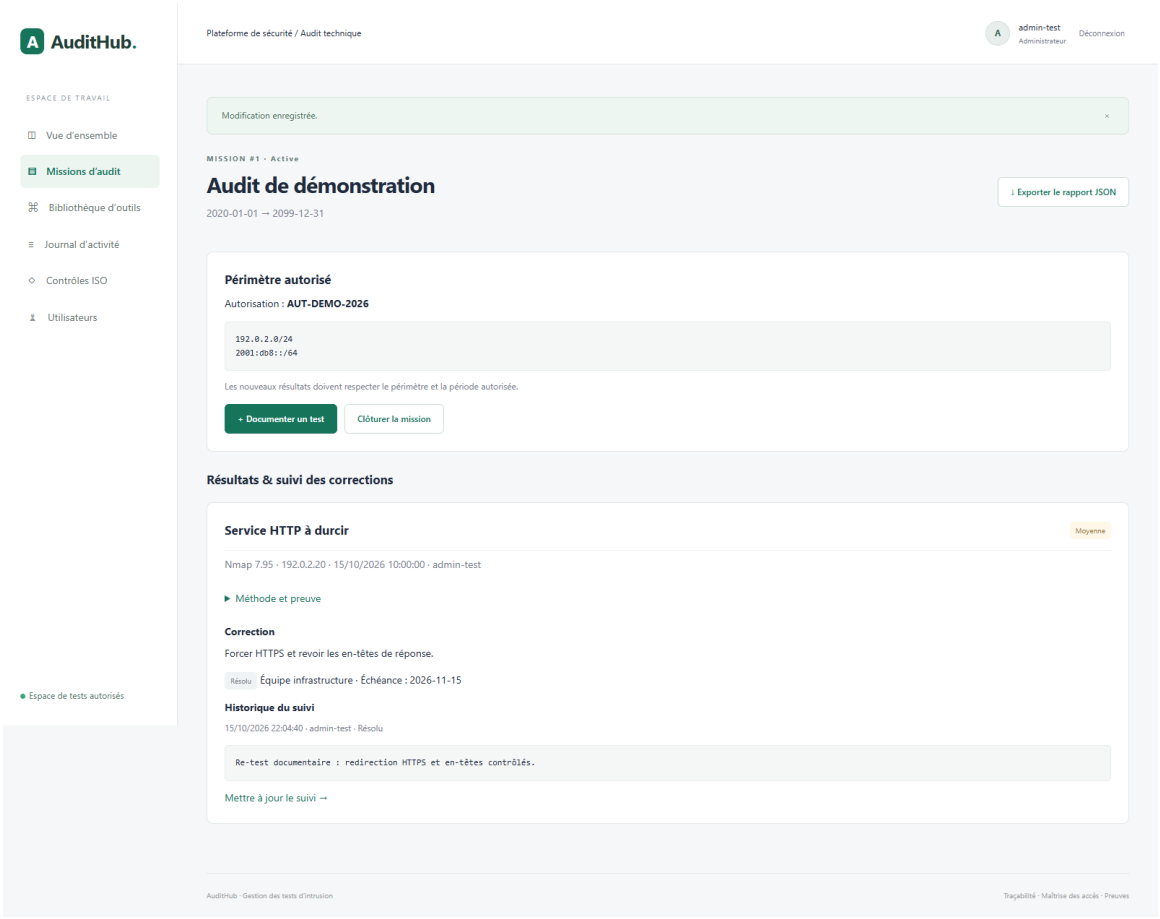
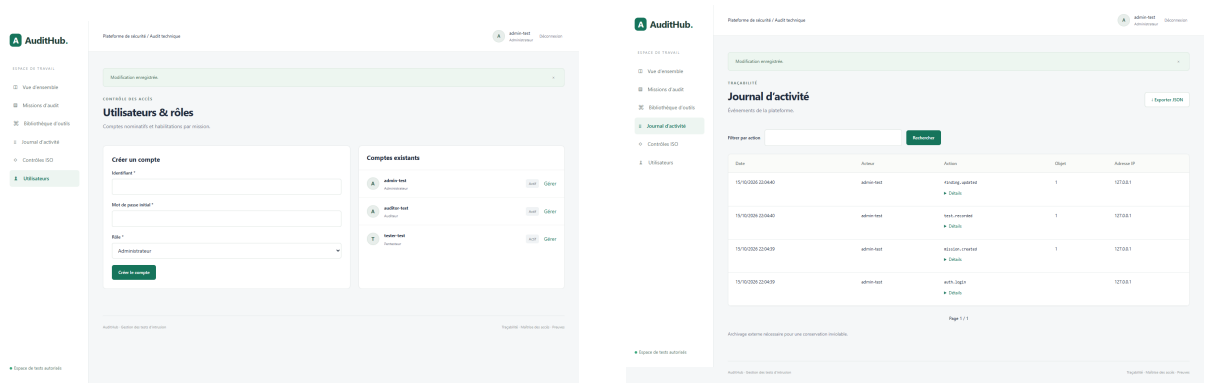


Figure 7.5 – Historique daté d'un suivi de correction

Sécurité implémentée

— mot de passe haché par Django et validateurs imposant notamment 12 caractères ;
— session de 30 minutes d'inactivité, expiration à la fermeture, cookie HttpOnly et SameSite Strict ;
— cookies Secure, redirection HTTPS et HSTS hors mode développement ;
— jeton CSRF renouvelé après connexion et mutations POST ;
— CSP sans script inline/eval, refus du framing, politique de permissions restrictive et `no-store` ;
— limitation par compte (5 échecs) et par IP (30 échecs) sur 15 minutes ; les succès ne consomment pas le quota ;
— filtrage des missions avant lecture, écriture et export ;
— preuve initiale non modifiable dans l'interface et empreinte SHA-256 ;
— journalisation des actions sensibles et historique des suivis.

Ces mesures ne garantissent pas l'inviolabilité face à un administrateur de base. L'empreinte stockée avec la preuve n'est pas une signature externe.



(a) Comptes et rôles **(b)** Journal d'activité

Figure 7.6 – Fonctions d'administration et de traçabilité

Chapitre 8

SYNTHÈSE DES CONSTATATIONS

8.1 Statistiques globales

La notion de non-conformité de certification n'est pas applicable faute d'audit organisationnel. La synthèse ci-dessous porte sur le logiciel et distingue les capacités validées des limites connues.

Table 8.1 – Indicateurs objectifs de la version finale

Indicateur	Valeur	Source
Tests backend exécutés	33/33	`manage.pytest`
Parcours navigateur exécutés	3/3	Playwright, Chrome headless
Captures de rapport	9	scénario isolé
Rôles applicatifs	3	`portal.security.ROLES`
Outils documentés	4	Nmap, Metasploit, Burp Suite, ZAP
Migrations applicatives	2	`portal/migrations`
Erreurs de check Django	0	modes développement et production
Migrations manquantes	0	`makemigrations--check`

8.2 Répartition par article / mesure

Table 8.2 – Contribution technique aux mesures ciblées

Mesure	Contribution d'AuditHub	Compléments organisationnels nécessaires
A.8.8	registre de constats, cible, sévérité, responsable, échéance, correction et re-test	inventaire d'actifs complet, veille, SLA, acceptation des risques, exceptions
A.8.25	RBAC, validation, sessions, tests automatisés, CI et guide de déploiement	politique de développement, revues indépendantes, SAST/SCA, séparation des environnements
A.8.29	méthode, outil/version, date, preuve, empreinte et historique de re-test	plan de recette signé, critères de sortie, environnement cible et validation indépendante

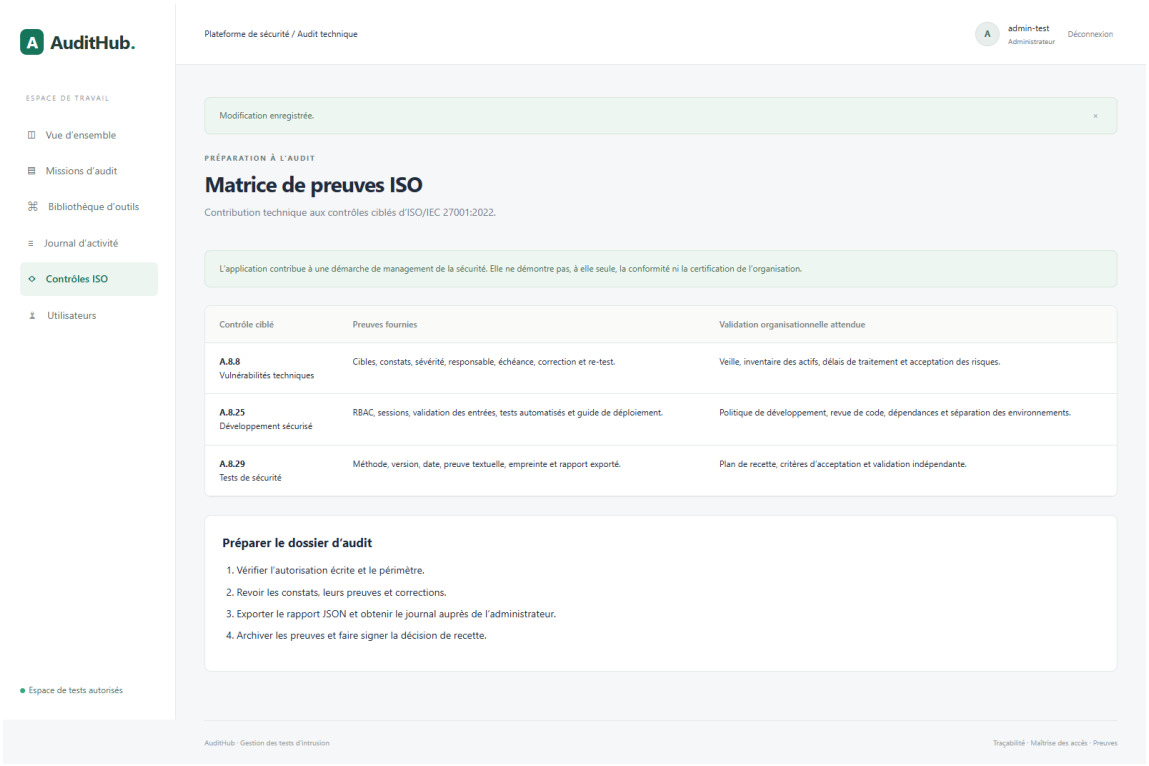


Figure 8.1 – Matrice ISO affichée par l'application, explicitement présentée comme contribution

Points forts identifiés

— séparation nette entre exécution offensive et gestion documentaire ;
— permissions serveur complétées par un cloisonnement au niveau des requêtes ;
— parcours de preuve cohérent, de l'autorisation à l'historique de correction ;
— gestion explicite des erreurs JSON et des erreurs de formulaires ;
— interface responsive sans ressource CDN et bundle reproductible via `npmci` ;
— tests de sécurité ciblant CSRF, XSS, RBAC, périmètre, dates, hachage et throttling ;
— documentation honnête sur les limites et absence de revendication de certification.

Chapitre 9

NON-CONFORMITÉS DÉTAILLÉES

Position de la revue

Aucune non-conformité ISO/IEC 27001 formelle n'est prononcée. Une telle qualification exigerait un organisme audité, un périmètre SMSI, des critères contractuels, des preuves organisationnelles et une autorité d'audit. La présente section recense plutôt les **écarts techniques et limites de mise en production** objectivement observables.

Écart technique ET-01 — persistance mono-instance

Critère	Exploitation fiable en contexte multi-utilisateur
Constat	La configuration utilise un fichier SQLite local avec un délai d'attente de 20 secondes.
Risque	Contention en ècriture, absence de haute disponibilité et sauvegarde dépendante du système de fichiers.
Preuve	`config/settings.py`, variable `DATABASES`.
Recommandation	Valider la concurrence attendue puis rendre la base configurable et migrer vers PostgreSQL si la charge ou la criticité l'exigent.

Écart technique ET-02 — journal non inviolable

Critère	Conservation indépendante des traces
Constat	Les `AuditEvent` sont stockés dans la même base que les données métier. Aucun endpoint ne les modifie, mais un administrateur système ou de base peut les altérer.
Risque	Perte ou modification de la preuve de traçabilité.
Recommandation	Exporter vers un stockage externe à rétention contrôlée et superviser les ruptures de collecte.

Écart technique ET-03 — identité et récupération de compte

Critère	Cycle de vie complet de l'identité
Constat	Les rôles et l'activation sont administrables et journalisés. Le changement de mot de passe reste une commande locale ; il n'existe ni récupération autonome, ni MFA, ni SSO.
Risque	Dépendance à l'administrateur système et protection d'identité limitée pour un contexte sensible.
Recommandation	Choisir un fournisseur d'identité selon le contexte, imposer MFA et définir un parcours de récupération audité.

Écart technique ET-04 — preuves uniquement textuelles

Critère	Conservation complète des résultats techniques
Constat	La preuve est un texte limité à 20 000 caractères. L'application ne reçoit pas les rapports natifs, captures ou pièces jointes.
Risque	Détails techniques incomplets et dépendance à un référentiel externe pour les originaux.
Recommandation	Si le besoin est validé, ajouter un stockage de fichiers isolé, antivirus, quotas, empreintes externes, contrôles d'accès et rétention.

Écart technique ET-05 — assurance de livraison partielle

Critère	Chaîne de livraison sécurisée
Constat	Une CI exécute build et tests ; les versions Python et npm sont verrouillées. Aucun SBOM, SAST/SCA, signature d'artefact ou scan DAST n'est fourni.
Risque	Vulnérabilités de dépendances ou d'implémentation non détectées automatiquement.
Recommandation	Ajouter analyse de dépendances, SAST, SBOM, contrôle des secrets et signature des livrables.

Écart technique ET-06 — exploitation à finaliser

Le dépôt documente Waitress et le proxy, mais ne fournit pas le proxy TLS, les certificats, un service système, une supervision, des alertes ni une preuve de restauration. Ces èléments dépendent de l'environnement cible et doivent être validés avant exposition réseau.

Table 9.1 – Priorisation proposée des limites

ID	Limite	Priorité	Horizon
ET-01	base et concurrence	Haute si production partagée	avant production
ET-02	journal externe	Haute pour valeur probatoire	avant production
ET-03	MFA/SSO/récupération	Moyenne à haute	selon risques
ET-04	pièces jointes	Moyenne	selon besoin
ET-05	assurance CI	Moyenne	prochaine itération
ET-06	exploitation	Haute	avant production

Chapitre 10

OBSERVATIONS ET OPPORTUNITÉS D'AMÉLIORATION

Observation O-01 — cohérence documentaire

La documentation expose clairement le mode documentaire et ses limites. Elle doit rester synchronisée avec les nombres de tests, la version du schéma d'export et les variables de proxy. La CI pourrait produire automatiquement un résumé de validation afin de réduire les informations datées.

Observation O-02 — accessibilité et navigateurs

Les formulaires utilisent labels, messages ARIA et ètats de chargement. Le navigateur automatisé couvre Chrome et un affichage mobile, mais aucun audit WCAG outillé ni test Firefox/WebKit n'est présent. Une campagne axe-core et des tests clavier constitueraient une amélioration mesurable.

Observation O-03 — score de risque

La sévérité est déclarative. L'ajout d'un vecteur CVSS validé, distinct de la priorité métier, permettrait de documenter le calcul sans remplacer le jugement de l'auditeur. Ce besoin n'est pas implémenté dans la version analysée.

Observation O-04 — intégration d'outils

Une extension d'exécution ne devrait pas être ajoutée au serveur web. La documentation propose à juste titre un worker isolé, une file de tâches, des profils autorisés, des quotas, un arrêt d'urgence et une validation du périmètre juste avant l'exécution. Cette orientation

est une opportunité future et non une capacité actuelle.

Bonne pratique BP-01 — défense en profondeur

Les contrôles d'interface ne sont jamais la seule barrière : permissions, filtrage des querysets, validation des formulaires, contraintes SQL, transactions et journalisation se complètent. Cette combinaison facilite la maintenance et limite l'impact d'un contournement du frontend.

Bonne pratique BP-02 — preuves reproductibles

Les captures de ce rapport sont reproductibles par le scénario Playwright fourni. Celui-ci lance une base temporaire, crée trois comptes synthétiques, remplit une mission et s'arrête sans toucher à `audit.sqlite3`. Cette approche garantit que les figures correspondent à l'application et non à des maquettes.

Chapitre 11

CONCLUSION ET RECOMMANDATION DE CERTIFICATION

11.1 Synthèse générale

AuditHub atteint son objectif de **prototype fonctionnel de gestion documentaire d'audits techniques**. La chaîne mission–autorisation–périmètre–constat–preuve–correction–re-test est cohérente, contrôlée par rôles et couverte par des tests. L'architecture Django/React de même origine est simple à déployer, et l'interface fournit des parcours utilisables sans serveur frontend distinct.

Les 33 tests Django et les 3 parcours navigateur réussissent dans l'environnement de validation. Le build React aboutit, les migrations sont cohérentes et le check Django ne signale pas de problème avec une configuration de production valide. Ces résultats prouvent les scénarios couverts, pas l'absence générale de vulnérabilités.

Le projet présente plusieurs qualités d'ingénierie : validations côté serveur, RBAC, transactions, contraintes SQL, historique de suivi, configuration par environnement, dépendances verrouillées, CI et documentation de déploiement. Ses limites sont explicites : SQLite, absence de journal WORM, MFA/SSO, stockage de fichiers, surveillance et tests indépendants.

Position relative à la certification

Aucune recommandation de certification ISO/IEC 27001 n'est èmise. AuditHub fournit des èléments qui peuvent contribuer à A.8.8, A.8.25 et A.8.29, mais l'évaluation d'un SMSI suppose les articles 4 à 10, la Déclaration d'Applicabilité, les risques, les politiques, les personnes, les processus et des preuves organisationnelles absentes de ce projet.

Dans un cadre académique, la version peut être considérée comme **livrable logiciel validé dans son périmètre documentaire**, sous réserve que l'évaluateur complète les informations institutionnelles et apprécie les choix au regard du sujet initial non fourni

dans le dépôt.

11.4 Prochaines ètapes

Table 11.1 – Feuille de route recommandée

Ètape	Responsable attendu	Condition de clôture
Compléter auteur, encadrant et contexte académique	porteur du projet	informations confirmées
Qualifier la charge et choisir la base	architecte / exploitation	essai de concurrence et sauvegarde
Déployer TLS, service et supervision	exploitation	recette de production documentée
Externaliser et protéger les journaux	sécurité / exploitation	rétention et restauration testées
Ajouter SAST/SCA/SBOM	èquipe de développement	pipeline bloquant et rapports archivés
Choisir MFA/SSO	responsable sécurité	authentification forte testée
Réaliser une revue indépendante	auditeur autorisé	rapport et actions suivies

Chapitre 12

ANNEXES

Annexe A : Liste des documents examinés

N°	Document	Usage
1	`README.md`	fonctions, installation, validation et limites
2	`docs/ARCHITECTURE.md`	architecture, menaces et déploiement
3	`docs/GUIDE_AUDITEUR.md`	rôles et processus opérationnels
4	`docs/RAPPORT_TESTS_ISO.md`	résultats de validation et matrice ISO
5	`Structurerapport.docx`	structure obligatoire et consignes de couleur
6	`requirements*.txt`, `package*.json`	dépendances et versions
7	`.github/workflows/ci.yml`	automatisation de validation

Annexe B : Inventaire du code examiné

Fichier / zone	Responsabilité
`portal/models.py`	entités, relations et contraintes
`portal/forms.py`	validation métier et comptes
`portal/views.py`	cas d'usage, transactions, exports
`portal/security.py`	rôles, journal, IP client et en-têtes
`portal/api.py`	sérialisation JSON et erreurs
`config/settings.py`	sessions, HTTPS, cookies, base, limites
`config/urls.py`	surface HTTP et coque React
`frontend/src/main.jsx`	navigation et pages

Fichier / zone	Responsabilité
`frontend/src/components.jsx`	composants, formulaires et constats
`frontend/src/api.js`	transport et erreurs
`portal/tests.py`	tests fonctionnels et de sécurité
`frontend/tests/workflows.spec.js`	tests bout en bout

Annexe C : Matrice des permissions

Action	Admin.	Pentesteur	Auditeur
Consulter les outils	Oui	Oui	Oui
Créer une mission	Oui	Non	Non
Consulter/exporter une mission	Toutes	Affectées	Affectées
Documenter un test	Oui	Affectées	Non
Suivre une correction	Oui	Affectées	Non
Clôturer/réouvrir	Oui	Non	Non
Créer/gérer un compte	Oui	Non	Non
Journal global/export	Oui	Non	Non
Journal personnel	Oui	Non	Oui

Annexe D : Endpoints principaux

Chemin API	Méthodes	Objet
`/api/session/`	GET	profil, rôle et jeton CSRF
`/api/connexion/`	GET, POST	authentification et limitation
`/api/missions/`	GET	missions autorisées, recherche et pagination
`/api/missions/nouvelle/`	GET, POST	création administrateur
`/api/missions/<id>/`	GET	détail et constats
`/api/missions/<id>/test/`	GET, POST	saisie de résultat
`/api/tests/<id>/suivi/`	GET, POST	suivi et historique
`/api/missions/<id>/rapport/`	GET	export JSON v2
`/api/journal/`	GET	consultation autorisée
`/api/journal/export/`	GET	export global administrateur
`/api/utilisateurs/`	GET, POST	comptes

Chemin	Méthodes	Objet
`/api/utilisateurs/<id>/acces/`	GET, POST	rôle et activation

Annexe E : Preuves documentaires collectées

1. sortie de 33 tests Django réussis ;
2. sortie de 3 parcours Playwright réussis ;
3. build esbuild terminé sans erreur ;
4. checks Django en développement et production sans erreur ;
5. vérification de migrations sans changement manquant ;
6. neuf captures sous `figures/screenshots/` ;
7. analyse de couleur du modèle sous `template_analysis.txt`.

Annexe F : Déclaration de non-responsabilité

Cette revue est fondée sur l'état disponible du dépôt et sur des tests automatisés. Elle ne garantit pas l'absence de vulnérabilité et ne remplace ni une recette de production, ni un test d'intrusion indépendant, ni un audit de certification.

Annexe G : Confidentialité

Le rapport est destiné à l'évaluation académique. Les captures reposent sur des identifiants et données synthétiques ; aucun mot de passe n'y apparaît. Toute diffusion ultérieure doit respecter la politique de l'établissement et les droits sur le code.

Signatures

Auteur du projet	Professeur / encadrant	Date
		15/10/2026

Bibliographie

[1] Django Software Foundation. Deployment checklist. https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/. Consulté le 15 octobre 2026.

[2] Django Software Foundation. Security in django. https://docs.djangoproject.com/en/5.2/topics/security/. Consulté le 15 octobre 2026.

[3] International Organization for Standardization. Iso/iec 27001 :2022 — information security management systems. https://www.iso.org/standard/27001. Consulté le 15 octobre 2026.

[4] International Organization for Standardization. Iso/iec 27002 :2022 — information security controls. https://www.iso.org/standard/75652.html. Consulté le 15 octobre 2026.

[5] Meta Open Source. React documentation. https://react.dev/. Consulté le 15 octobre 2026.